

GauzMem: 让 agent 不只是搜索，而是会联想。

0.2 已完成 query-driven memory core。0.3 先做 manual / offline dreaming prototype；长期方向是让图在真实请求之外继续生长。

目标听众：第一次参与 GauzMem 的团队成员。

为什么做

关键词和 grep 很强；缺的是“应该想到”。

大型项目里，agent 往往能解决眼前问题，却不知道已有模块、旧约束、旧偏好和当前任务有关。

记忆不是更大的检索框

普通搜索

命中关键词，读上下文，再继续搜。

GauzMem

记录 evidence 之间为什么值得互相联想到。

先对齐词

六个基础对象

先记住这六个对象，后面的图就能读懂。

source

XiaoBa session log 或已注册 attachment text。

evidence node

从 source 抽出的 exact quote，不是模型摘要。

whyRelevant edge

0.2 的 edge type 是 localAssociation；whyRelevant 是边上的说明文本，不是事实本身。

run

0.2: retrieve / tool search / sweep。0.3 planned: dream episode。

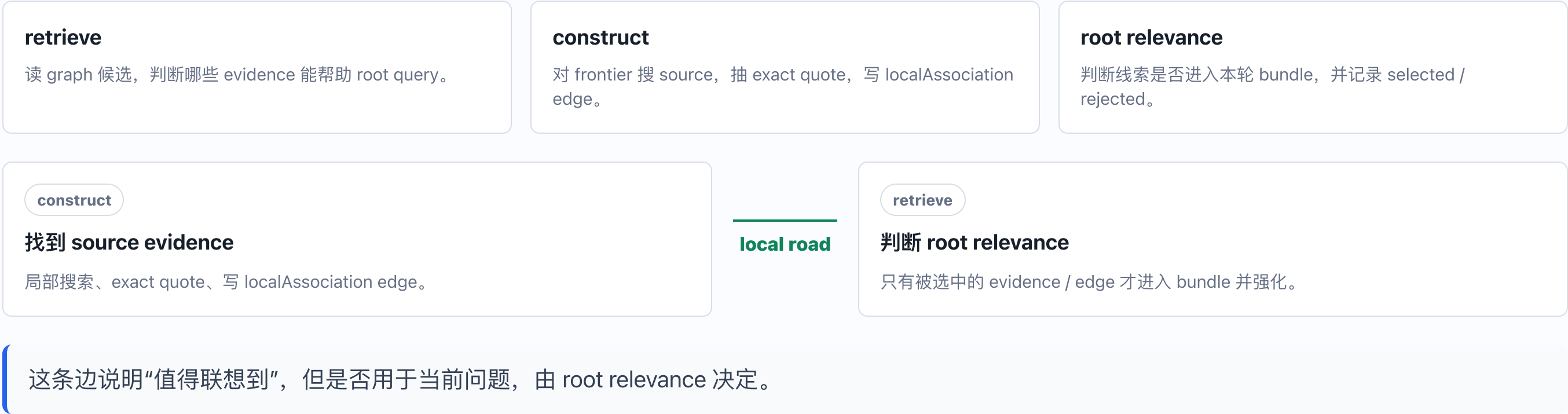
bundle

临时注入 agent 的自然语言记忆线索。

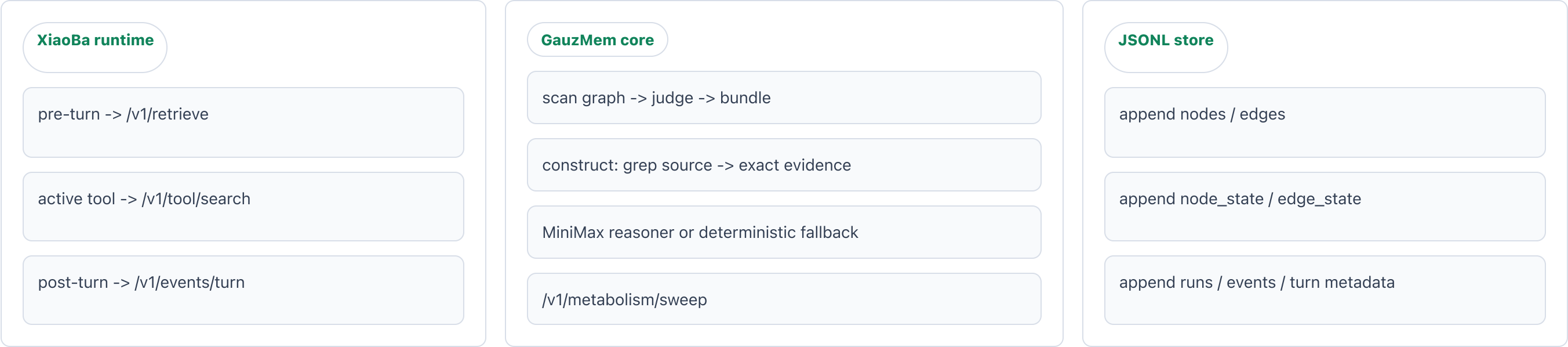
state

权重、可见性、tick、selected / rejected / decay 记录。

Retrieve 决定“对当前问题是否有用”； Construct 只修本地联想路。



XiaoBa 只发 query / event；GauzMem sidecar 负责记忆内部状态。



部署约束：sidecar 持有 storeRoot、LLM key、source roots 和 callType 策略；caller 只提交本轮意图。

从用户问题到临时 bundle，0.2 通常先走 graph，再按需 construct。



这不是每轮必经的线性流程。Construct 触发条件包括冷启动、graph seed 未选中、没有新增有效 route、当前 selected graph 不足且有 source roots。

两个 evidence 不相似，也可能应该互相联想到。



用户的代码语言偏好发生更新；近期 Python 偏好更该被优先参考。

AGENT 看到什么

给 agent 的是记忆线索，不是调试记录。

sourceRef、runId、权重、trace 留在 JSONL 和 dashboard。上下文里只放可读 evidence 与 whyRelevant。

store 保留 exact quote；bundle 只做轻量排版，不新增事实、不暴露 IDs。

PROMPT STAYS SMALL

```
[gauzmem_recall]
相关记忆线索：

- 最近，用户说现在更喜欢 Python。
  可能联想到：
  - 一年前，用户说自己喜欢 C：
    用户的代码语言偏好发生更新；近期 Python 偏好更该被优先参考。
[/gauzmem_recall]
```

遗忘按 agent 的认知 tick 发生，不按墙上时间。

信号	当前含义	注意
selected	真实 query 中被 root relevance 选中。	强强化，增加普通 selectedCount。
rejected	root relevance 判断无关或误导。	降权，尤其是 edge。
effectiveWeight	运行时根据 weight、selectedCount、lastTouchedTick 计算。	不直接改 numeric weight。
sweep	显式物化 active -> faint 或 faint -> dormant。	写 decayed event，dashboard 可复盘。

Proposal

如果只靠真实 query，联想图可能长得太慢。

0.3 先做手动 / 同步的 dream prototype：从已有 evidence 自我发问，验证能否提前补出有用联想边。

要验证

后续真实 query 是否更快命中相关线索。

要防止

内部梦境自我强化，污染真实 recall。

Dreaming 仍然做 root relevance； 否则就是盲目建图。



第一版 dreaming 要保守：手动、同步、可复盘。

做	暂不做	原因
CLI / API 手动触发 dream。	自动后台 daemon。	先看污染、成本和收益。
从 retrievable non-dormant node 选 seed。	dormant recovery。	复活策略需要单独设计。
dream_created 低初始权重。	从 dream-created node 继续 dream。	避免内部图自我膨胀。
Dreaming 独立 dashboard 区域。	混进 Recall Replay。	区分用户会话和内部 episode。

安全规则：dream-created 是梦境中发现的 source evidence，不是生成摘要；whyRelevant 是联想说明，不是事实声明。

这些是实现入口，不是新概念；建议按依赖顺序推进。

方向	第一批改动	验收方式
1. Core	initialNodeIds、bundleTarget、dream state profile。	initialNodeIds 只激活起点；bundleTarget=internal 不返回 prompt bundle；dream_selected 不增加普通 selectedCount。
2. Dream policy	seed picker、generateDreamQuestions、dry-run plan。	同一 store 能输出稳定 seeds/questions；dry-run 不写 nodes/edges/state。
3. API / CLI / UI	POST /v1/dream、gauzmem dream、dashboard Dreaming 区域。	dream runs 进独立区域；不混进用户 Recall Replay。
4. Evaluation	dream 前后真实 query 对比。	命中率、无关 bundle 率、成本、图增长都有记录。

0.3 成功标准：dream 长出的边，后续真实 query 能命中，并减少 agent 主动搜索成本。